

SISTEMAS OPERATIVOS

08011442

Germán Sánchez Torres, I.S., M.Sc., Ph.D.

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com



Señales

Comunicación asíncrona entre procesos

Contenido

1. Qué es una señal y quién la genera
2. Señales estándar
3. Acciones por defecto
4. Generación, pendiente y entrega
5. Captura con signal y sigaction
6. Entrega y manejo de una señal
7. Enviar señales: kill
8. Temporización: alarm
9. Esperar señales: pause y sleep
10. Ejemplos
11. Ejercicios



Qué es una señal

Qué es una señal y quién la genera

- Una **señal** es un evento que genera el sistema UNIX/Linux en respuesta a alguna condición.
- Al recibirla, el proceso puede tomar alguna acción (la veremos en unas diapositivas).
- Se genera por:
  - Condiciones de error: violación de segmento de memoria, error de punto flotante, instrucción ilegal…
  - El shell y los manejadores de terminal (por ejemplo, `Ctrl`+`C`).
  - Un proceso, para notificar un evento a otro.

En los tres casos, la **interfaz de programación** que usa el proceso para reaccionar es la misma.

El catálogo de señales: <signal.h>

- <signal.h> declara todas las señales posibles; salvo excepciones, su acción por defecto es terminar el proceso.
- Todos los nombres de señal empiezan por SIG (SIGINT, SIGKILL, SIGTERM, ...).
- Linux soporta todas las señales definidas en POSIX.1.

Nombre y número de cada señal en Linux (x86-64):

```
$ kill -l
 1) SIGHUP     2) SIGINT     3) SIGQUIT    4) SIGILL     5) SIGTRAP
 6) SIGABRT    7) SIGBUS     8) SIGFPE     9) SIGKILL   10) SIGUSR1
11) SIGSEGV   12) SIGUSR2   13) SIGPIPE   14) SIGALRM   15) SIGTERM
16) SIGSTKFLT 17) SIGCHLD   18) SIGCONT   19) SIGSTOP   20) SIGTSTP
...
```

Catálogo completo con descripciones: `man 7 signal`. Algunos números cambian entre arquitecturas: en los programas use siempre el nombre (SIGUSR1), no el número.

Señales estándar: generales y de error

Señal	Descripción	Acción por defecto
SIGHUP	Terminal colgada, o terminó el proceso controlador	Terminar
SIGINT	Interrupción desde el teclado (`Ctrl+C`)	Terminar
SIGQUIT	Salida desde el teclado (`Ctrl+\`)	Terminar + volcado
SIGILL	Instrucción ilegal	Terminar + volcado
SIGABRT	Fin anormal del proceso (`abort`)	Terminar + volcado
SIGFPE	Excepción de punto flotante (por ejemplo, dividir por 0)	Terminar + volcado
SIGKILL	Terminar el proceso — no se puede capturar ni ignorar	**Terminar**
SIGSEGV	Acceso a una posición de memoria inválida	Terminar + volcado
SIGPIPE	Escritura en una tubería sin lectores	Terminar
SIGALRM	Reloj de alarma (`alarm`)	Terminar
SIGTERM	Solicitud de terminación (la que envía `kill` por defecto)	Terminar
SIGUSR1	Señal definida por el usuario 1	Terminar
SIGUSR2	Señal definida por el usuario 2	Terminar

Señales estándar de control de trabajos

- Estas señales coordinan la **shell** y los procesos en primer y segundo plano (control de trabajos):

Señal	Descripción	Acción por defecto
SIGCHLD	Un proceso hijo se detuvo o terminó	Ignorar
SIGCONT	Continuar la ejecución, si estaba detenido	Continuar
SIGSTOP	Detener el proceso — no se puede capturar ni ignorar	**Detener**
SIGTSTP	Señal de detención desde la terminal (`Ctrl+Z`)	Detener
SIGTTIN	Un proceso en segundo plano intenta leer de la terminal	Detener
SIGTTOU	Un proceso en segundo plano intenta escribir en la terminal	Detener

SIGKILL y SIGSTOP son las dos únicas señales que ningún proceso puede capturar, bloquear ni ignorar: garantizan que siempre se pueda terminar o detener un proceso.

Acciones por defecto ante una señal

- **Ignorar**: el kernel descarta la señal; el proceso nunca se entera de que ocurrió.
- **Terminar** (*terminated*): el proceso finaliza; se llama terminación anormal, a diferencia de la que provoca `exit()`.
- **Terminar con volcado de memoria** (*core dump*): además de terminar, se genera un archivo con la imagen de memoria del proceso, útil para depurar.
- **Detener** (*stopped*): se suspende la ejecución del proceso.
- **Continuar** (*resumed*): el proceso reanuda su ejecución tras haber estado detenido.

Qué se puede configurar ante una señal

- Además de dejar la acción por defecto, un proceso puede cambiar la **disposición** de una señal (qué ocurre cuando llega). Hay tres opciones:

`SIG_DFL`
Acción por defecto
Restaura la acción por defecto; sirve para deshacer un cambio anterior.

`SIG_IGN`
Ignorar la señal
Útil cuando la acción por defecto sería terminar el proceso.

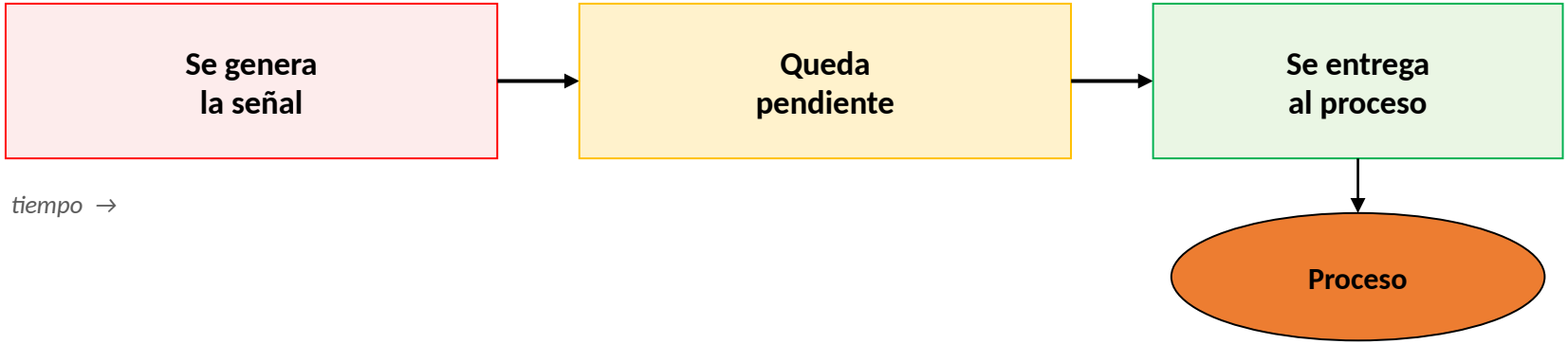
`manejador`
Ejecutar un manejador
Una función propia (signal handler) que se invoca al llegar la señal.

`SIGKILL` y `SIGSTOP` son la excepción: su disposición no se puede cambiar.



Captura
y manejo de señales

Generación, pendiente y entrega de una señal



Entre el momento en que se genera la señal y el momento en que se entrega, la señal permanece pendiente.

- Las señales pendientes se entregan al proceso tan pronto como este es planificado para ejecutarse, o de inmediato si ya está en ejecución.

Capturar una señal: *signal* y *sigaction*

- Con la función `signal` (ISO C) se indica qué función del programa capturará una señal específica.
- La llamada al sistema `sigaction` (POSIX) hace lo mismo que `signal`, con más opciones y con un comportamiento más portable entre variantes de Unix.
- El resto del programa no cambia: solo cambia qué ocurre cuando llega esa señal en concreto.

En los ejemplos de este tema se usa `signal`; `sigaction` es la alternativa recomendada en programas reales.

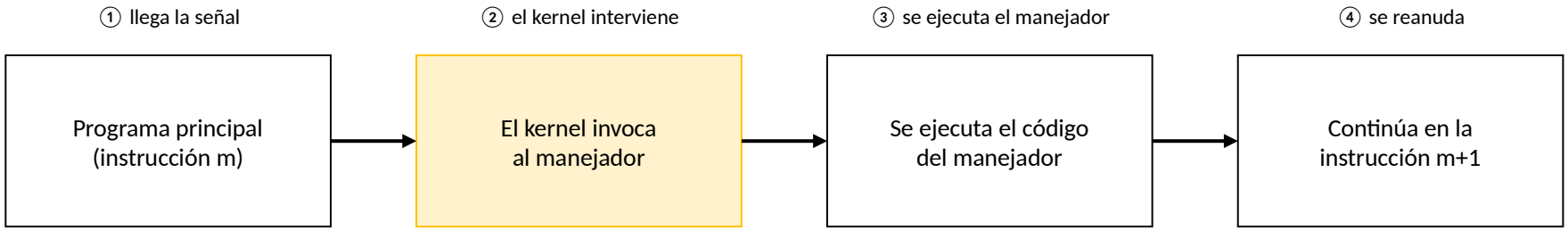
Capturar una señal: *signal*

void (*signal(int sig, void (*func)(int)))(int);

Instala `func` como la función que se ejecutará cuando llegue la señal `sig`.

Include	`<signal.h>`
Retorna	**Éxito:** la dirección del manejador que estaba instalado antes. **Falla:** `SIG_ERR` y `errno`.
Argumentos	• ***int sig***: la señal que se quiere capturar. • ***void (*func)(int)***: la función manejadora; recibe el número de señal como único argumento y no retorna nada (`void`).
Nota	`func` también puede ser `SIG_IGN` (ignorar la señal) o `SIG_DFL` (restaurar el comportamiento por defecto), en lugar de una función propia.

Entrega y manejo de una señal



- El proceso ejecuta su código con normalidad hasta que llega la señal (①).
- El kernel suspende esa ejecución (②) e invoca al manejador instalado con `signal` para esa señal.
- El código del manejador se ejecuta (③) y, al retornar, el kernel reanuda el programa principal justo en el punto donde fue interrumpido (④), con la instrucción siguiente.

Ejemplo: capturar *SIGINT*

```c
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

volatile sig_atomic_t senal = 0;

void manejador(int sig) {
    senal = sig;        /* solo anota que llegó */
}

int main(void) {
    signal(SIGINT, manejador);
    while (1) {
        if (senal) {
            printf("Capturada señal %d\n", (int)senal);
            senal = 0;
        }
        printf("Hello World!\n");
        sleep(1);
    }
}
```

```
$ ./sigint
Hello World!
Hello World!
^CCapturada señal 2
Hello World!
Hello World!
^CCapturada señal 2
Hello World!
^\Quit (core dumped)
```

- `Ctrl+C` envía `SIGINT`: como hay manejador, el programa **no termina**.
- Para terminarlo: `Ctrl+\` (`SIGQUIT`) o `kill -9`.

El manejador solo anota la señal en una variable `volatile sig_atomic_t`; el mensaje lo imprime `main`. Dentro de un manejador solo deben usarse funciones *async-signal-safe*, y `printf` no lo es.



Enviar y esperar señales

Enviar una señal: *kill*

int kill(pid_t pid, int sig);

Envía la señal `sig` al proceso (o grupo de procesos) identificado por `pid` (`sig` = 0 no envía nada; solo comprueba permisos).

Include	`<signal.h>`
Retorna	**Éxito:** 0. **Falla:** -1 y `errno` (EPERM, ESRCH).

pid	Envía la señal a...
`> 0`	el proceso con ese pid exacto
`0`	todos los procesos del mismo grupo que el emisor
`-1`	todos los procesos para los que hay permiso (salvo init)
`< -1`	todos los procesos del grupo |pid|

Para enviar una señal a otro proceso, el UID real o efectivo de quien envía debe coincidir con el UID real o efectivo del receptor (o ser `root`).

Temporización: *alarm*

unsigned int alarm(unsigned int seconds);

Programa el envío de `SIGALRM` al propio proceso, transcurridos `seconds` segundos.

Include	`<unistd.h>`
Retorna	Segundos que quedaban de una alarma previa (`0` si no había ninguna activa).
Argumentos	• ***unsigned int seconds***: segundos hasta la alarma; `0` desactiva una alarma previamente activa, si la había.
Nota	Solo puede haber **una** alarma pendiente por proceso: una nueva llamada reemplaza a la anterior (no las acumula).

Esperar una señal: *pause* y *sleep*

int pause(void);

Include	`<unistd.h>`
Retorna	Solo retorna cuando se captura una señal y su manejador termina: -1 con `errno = EINTR`. Nunca retorna "con éxito".

unsigned int sleep(unsigned int seconds);

Include	`<unistd.h>`
Retorna	Segundos que quedaban por dormir si una señal lo interrumpió antes; `0` si durmió el tiempo completo.

Si la señal que llega tiene como acción terminar el proceso, ni `pause` ni `sleep` retornan: el proceso termina.

Ejemplo: *alarm* y *pause*

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

volatile sig_atomic_t recibida = 0;

void manejador(int sig) { recibida = sig; }

int main(void) {
    signal(SIGALRM, manejador);
    alarm(2);            /* SIGALRM dentro de 2 s */
    printf("Esperando la alarma...\n");
    pause();             /* bloquea hasta una señal */
    printf("Señal %d recibida; fin\n", (int)recibida);
    return 0;
}
```

```
$ ./alarma
Esperando la alarma...
Señal 14 recibida; fin
```

- `alarm(2)` programa un `SIGALRM` para dentro de 2 segundos y retorna de inmediato.
- `pause()` bloquea el proceso; al llegar `SIGALRM` se ejecuta el manejador y `pause()` retorna.
- Sin el manejador, la acción por defecto de `SIGALRM` terminaría el proceso.

Generar una señal en el propio proceso: *raise*

```c
#include <stdio.h>
#include <signal.h>

/* raise(signo) equivale a kill(getpid(), signo) */
int main(void) {
    int sig;
    printf("Ingrese la señal a generar\n");
    fflush(stdout);
    scanf("%d", &sig);
    printf("Generando la señal %d\n", sig);
    fflush(stdout);  // por si la señal termina el proceso
    raise(sig);
    printf("Señal generada\n");
    return 0;
}
```

- `raise(sig)` (ISO C, `<signal.h>`) equivale a `kill(getpid(), sig)`.
- Con `sig = SIGINT` (2) la acción por defecto termina el proceso: nunca se imprime *"Señal generada"*.
- El código 130 = 128 + 2 indica que el proceso terminó por la señal 2.

```
$ ./raise
Ingrese la señal a generar
2
Generando la señal 2
$ echo $?
130
```

Ejemplo: sincronizar dos procesos con *SIGUSR1*

```c
#include <stdio.h>
#include <signal.h>
#include <sys/wait.h>
#include <unistd.h>

void manejador(int sig) { (void)sig; }

int main(void) {
    pid_t c;
    signal(SIGUSR1, manejador);
    if (!(c = fork())) {
        pause();
        printf("Hijo\n");
    } else {
        printf("Padre\n");
        fflush(stdout);
        kill(c, SIGUSR1);
        wait(NULL);
    }
    return 0;
}
```

```
$ ./sigusr1
Padre
Hijo
$ ./sigusr1
Padre
^C
```

Dos ejecuciones del mismo programa: la segunda se quedó bloqueada y hubo que interrumpirla con Ctrl+C.

Carrera clásica de `pause()`: si `SIGUSR1` llega antes de que el hijo ejecute `pause()`, el manejador corre antes, `pause()` espera una señal que ya pasó y ambos procesos quedan bloqueados. El resultado depende del planificador.

Solución robusta: bloquear la señal con `sigprocmask` antes de `fork` y esperarla con `sigsuspend`, que la desbloquea y espera en un solo paso. El siguiente ejemplo la aplica.

Ejemplo: contador de señales entre padre e hijo

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

void manejador(int sig) { (void)sig; }

int main(void) {
    pid_t c;
    int n, i;
    sigset_t usr1, vacio;
    signal(SIGUSR1, manejador);
    sigemptyset(&vacio);
    sigemptyset(&usr1);
    sigaddset(&usr1, SIGUSR1);
    /* bloqueada: si llega antes de
       esperarla, queda pendiente */
    sigprocmask(SIG_BLOCK, &usr1, NULL);
    printf("Digite n=");
    fflush(stdout); /* antes de fork() */
    if (scanf("%d", &n) != 1) return 1;
    if (!(c = fork())) {
        for (i = 0; i < n; i++) {
            sigsuspend(&vacio); /* su turno */
            printf("Hijo\n");
            kill(getppid(), SIGUSR1);
        }
    } else {
        for (i = 0; i < n; i++) {
            printf("Padre\n");
            kill(c, SIGUSR1);
            sigsuspend(&vacio);
        }
        wait(NULL);
    }
    return 0;
}
```

```
$ ./pingpong
Digite n=3
Padre
Hijo
Padre
Hijo
Padre
Hijo
```

Cada proceso alterna `kill()` (despierta al otro) y `sigsuspend()` (espera su turno), `n` veces. `SIGUSR1` está bloqueada desde antes de `fork()`: si llega antes de tiempo queda pendiente, y `sigsuspend()` la desbloquea y espera en un solo paso. Ninguna señal se pierde.

Ejercicios

- **Ejercicio 1.** Escribir un programa que, usando `alarm`, ejecute `system("date")` cada 2 segundos. Mientras se ejecuta esa llamada, debe ignorarse cualquier `Ctrl+C` recibido.
- **Ejercicio 2.** Escribir un programa en el que un proceso padre temporice a un hijo: el padre crea un hijo que ejecuta un comando recibido como argumento y espera su finalización; si el hijo no termina en un tiempo dado, el padre le envía la señal `SIGKILL`.

Resumen

- Una **señal** es un evento asíncrono generado por el kernel, la terminal o un proceso; se entrega con la misma interfaz sin importar el origen.
- Cada señal tiene una acción por defecto (ignorar, terminar, terminar con volcado, detener, continuar); `SIGKILL` y `SIGSTOP` no se pueden capturar ni ignorar.
- Entre que se genera y se entrega, la señal está **pendiente**; `signal()`/`sigaction()` permiten sustituir la acción por defecto por un manejador propio.
- `kill()` envía una señal a un proceso; `alarm()` programa un `SIGALRM` futuro; `pause()`/`sleep()` bloquean al proceso hasta que llega una señal.
- Un manejador debe hacer lo mínimo (por ejemplo, anotar en una variable `volatile sig_atomic_t`): `printf` no es *async-signal-safe*.
- `pause()` tiene una condición de carrera clásica: si la señal llega antes de llamarla, el proceso puede quedar bloqueado para siempre. Se evita con `sigprocmask()` + `sigsuspend()`.

Referencias y lecturas

- Kerrisk, M. *The Linux Programming Interface*, caps. 20 a 22 (señales). No Starch Press, 2010.
- Stevens, W. R. y Rago, S. A. *Advanced Programming in the UNIX Environment*, cap. 10 (Signals). Addison-Wesley.
- Silberschatz, A., Galvin, P. y Gagne, G. *Operating System Concepts*, comunicación entre procesos.
- Páginas de manual: `man 7 signal`, `man 2 kill`, `man 2 sigaction`, `man 3 signal`, `man 2 alarm`, `man 2 pause`, `man 2 sigsuspend`.